PHASE

0

Foundations and Toolchain

Set up the workshop: a cross compiler, an emulator, a debugger, and the C habits a kernel needs

Overall progress toward Turk-OS 1.0 (this phase in red) 0 % → 5 %

Duration	2–3 weeks
Difficulty	★☆☆☆☆ (1/5)
You need	Basic C, a Fedora Linux machine, curiosity
You will have	A working `i686-elf` cross toolchain, QEMU and GDB, a Git repository with the Turk-OS skeleton, and a small C library (strings, bitmaps, lists, ring buffer) with unit tests. You will reuse that library inside the kernel.

1 Why this phase

An operating system kernel is a C program with nothing underneath it: no standard library, no `printf`, no `malloc`, no operating system to catch its mistakes. Before you can write one, you need three things. The first is a **toolchain** that produces code for a bare x86 machine instead of for Linux. The second is a **laboratory**, an emulator you can reboot a hundred times an hour with a debugger attached. The third is **C fluency at the level of bits and addresses**: pointers to fixed memory locations, bit masks, struct layout, and the build pipeline from `.c` file to executable.

This phase builds all three and gives you the vocabulary of the field. It is the only phase with no kernel code, and the temptation is to rush it. Don't. Almost every mysterious failure in Phases 1–3 comes from a toolchain misconfiguration or a C misunderstanding that could have been fixed here in ten minutes.

2 Concepts you need

2.1 What an operating system is

OSTEP describes the OS through three ideas that also organise the whole book and this plan. It **virtualizes** the hardware, so each program believes it has its own CPU and its own memory (Phases 5–11). It manages **concurrency**, so many activities can safely share the machine (Phases 8–9). It provides **persistence**, so data survives power loss (Phases 12–13). MOS gives two complementary views of the same thing: the OS as an *extended machine* that hides ugly hardware behind clean abstractions, and as a *resource manager* that shares the CPU, memory and devices fairly and safely.

Kernel mode and user mode

The CPU has at least two privilege levels. Code in **kernel mode** (ring 0 on x86) can do anything: touch any memory, talk to devices, disable interrupts. Code in **user mode** (ring 3) can't, and if it tries, the CPU stops it and hands control to the kernel. This hardware-enforced split is what lets an OS protect itself and its programs from each other (OSC §1.4). For the first nine phases everything you write runs in ring 0. Ring 3 arrives in Phase 10.

2.2 The machine you are going to program

Read MOS §1.3 with this picture in mind. The **CPU** fetches instructions from memory and executes them, keeping its working values in a handful of **registers**. **Memory** is one big array of bytes, indexed by address. **Devices** (screen, keyboard, disk, timer) are controlled by reading and writing their registers, either through a separate I/O address space with special `in`/`out` instructions, or by mapping them into ordinary memory addresses. When a device needs attention it raises an **interrupt**, which makes the CPU pause what it is doing and jump to a handler the OS registered earlier. Nearly everything in Phases 2–4 is a concrete case of these four sentences.

2.3 C the way a kernel uses it

A kernel is compiled **freestanding**: there is no C library, but the compiler still provides the headers that only define types and macros: `stdint.h`, `stddef.h`, `stdbool.h`, `stdarg.h`, `limits.h` and a few others. Everything else you write yourself. The listing shows the idioms you will use every day. Make sure you can explain each line before moving on.

idioms.c (study this, you will write code like it constantly)

```c
#include <stdint.h>
#include <stddef.h>

/* 1. A pointer to a fixed physical address: the VGA text buffer.            */
/*    volatile = "every read/write really happens"; the compiler may not     */
/*    cache, merge or delete these accesses.                                  */
volatile uint16_t *const vga = (volatile uint16_t *)0xB8000;

/* 2. Bit manipulation: set, clear, test bit n of a flags word.              */
#define BIT(n)            (1u << (n))
#define SET(x, n)         ((x) |=  BIT(n))
#define CLEAR(x, n)       ((x) &= ~BIT(n))
#define TEST(x, n)        (((x) & BIT(n)) != 0)

/* 3. Alignment: round an address up/down to a power-of-two boundary.        */
#define ALIGN_DOWN(x, a)  ((x) & ~((uintptr_t)(a) - 1))
#define ALIGN_UP(x, a)    ALIGN_DOWN((x) + (a) - 1, (a))

/* 4. A struct whose memory layout is dictated by hardware: no padding.      */
struct __attribute__((packed)) gdt_ptr {
    uint16_t limit;
    uint32_t base;
};                        /* sizeof == 6, not 8 */

/* 5. Function pointers: tables of handlers, e.g. one per interrupt.         */
typedef void (*handler_t)(void);
static handler_t handlers[256];
```

2.4 From source file to bootable image

You will spend a lot of time looking at what the toolchain produced, so learn the pipeline now (Figure 1). The compiler turns C into assembly and the assembler turns that into an **object file** containing **sections**: `.text` (code), `.rodata` (constants), `.data` (initialised globals) and `.bss` (zero-initialised globals, which take no space in the file). The **linker** combines object files and places each section at the address a **linker script** tells it to. The result is an **ELF** executable that GRUB or QEMU can load. OSC §2.5 covers linkers and loaders.

2.5 Numbers you will see every day

Kernel code talks in hexadecimal and powers of two. Memorise this table; you will recognise these values in register dumps long before you understand everything around them.

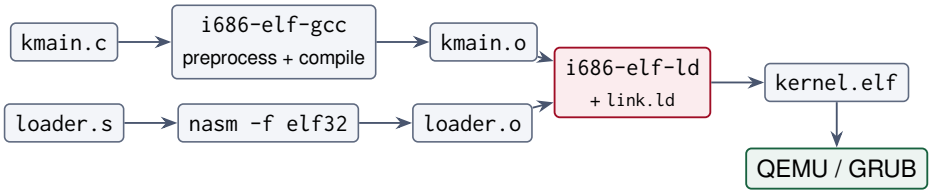


Figure 1: The build pipeline you set up in this phase and use for the rest of the project.

Value	Hex	Where you meet it
512 bytes	`0x200`	one disk sector, the boot sector (Phase 1)
4 KiB	`0x1000`	one page or page frame (Phases 5–6)
1 MiB	`0x100000`	where the kernel is loaded (Phase 1)
4 MiB	`0x400000`	memory covered by one page table (Phase 6)
3 GiB	`0xC0000000`	start of the higher-half kernel (Phase 6)
4 GiB	`0x100000000`	the whole 32-bit address space

3 Reading plan

Read the REQUIRED items before starting the tasks. The RECOMMENDED items make the tasks easier and are worth the time this early. Your reading order: OSTEP first, then MOS §1.3, then LOB.

Priority	Book	What to read	Pages	Why now
REQUIRED	OSTEP	Ch. 1 *A Dialogue on the Book*; Ch. 2 *Introduction to Operating Systems*	1–22	The three big ideas; the map for the whole plan.
REQUIRED	MOS	§1.3 *Computer Hardware Review* (processors, memory, I/O devices, buses, booting)	20–35	The hardware you will program directly.
REQUIRED	LOB	Ch. 1 *Introduction*; Ch. 2 up to *Booting* (Tools, Quick Setup, Build System, Virtual Machine)	7–12	How the practical guide works and what it expects.
RECOMMENDED	MOS	§1.1 *What Is an OS?*; §1.5 *OS Concepts*	3–6, 38–50	Vocabulary: processes, address spaces, files, shell.
RECOMMENDED	MOS	§1.8 *The World According to C*	73–77	Header files, large projects, the C runtime model.
RECOMMENDED	OSC	Ch. 1 §1.1–1.4 (organization, interrupts, storage, dual-mode)	4–27	A second, more detailed view of the same ground.
OPTIONAL	OSC	§1.9 *Kernel Data Structures*; §2.5 *Linkers and Loaders*	36–40, 75–77	Background for the warm-up library and the build pipeline.
OPTIONAL	OSDI	§1.1–1.2 (what an OS is, history, history of MINIX 3)	4–19	Where MINIX came from and why it matters for teaching.
RECOMMENDED	LKD	Ch. 2, *A Beast of a Different Nature*: no libc, GNU C, no memory protection, no floating point, a small fixed stack	16–21	Why kernel C is not ordinary C, before you write the warm-up library.
OPTIONAL	LKP	Ch. 1, *Static analysis tools for the Linux kernel*	34–35	Tools such as sparse and cppcheck you can point at your own code.

4 Build it

Task 0.1 Install the host tools

Everything below is packaged in Fedora. `grub2-tools-extra` provides `grub2-mkrescue` (Phase 1) and `mtools` is used by it. The second line installs what GCC needs to build from source.

terminal (host)

```
sudo dnf install gcc make git nasm qemu-system-x86 gdb xorriso mtools \
                 grub2-tools-extra grub2-pc-modules
sudo dnf install bison flex gmp-devel mpfr-devel libmpc-devel texinfo
```

Verify: `nasm -v`, `qemu-system-i386 --version` and `gdb --version` all print a version.

Task 0.2 Make your editor's terminal see those tools

Your VS Code is the Flatpak build, so its integrated terminal runs inside a sandbox where host programs like `nasm` and `qemu-system-i386` are not visible. Either work in a normal terminal window, or add a terminal profile to VS Code's `settings.json` that starts a shell on the host:

settings.json (VS Code)

```
"terminal.integrated.profiles.linux": {
  "host-bash": { "path": "/usr/bin/flatpak-spawn", "args": ["--host", "--env=TERM=xterm-256color", "bash"] }
},
"terminal.integrated.defaultProfile.linux": "host-bash"
```

Verify: In a new VS Code terminal, `which nasm` prints `/usr/bin/nasm`.

Task 0.3 Build the i686-elf cross compiler

The Fedora `gcc` produces programs for Linux. It assumes things that are false inside your kernel: that a C library exists, that the stack protector runtime is available, that code should be position-independent. A **cross compiler** targeting `i686-elf` assumes nothing. Follow the OSDev Wiki page *GCC Cross-Compiler*. Download recent binutils and GCC sources from `https://ftp.gnu.org/gnu/`, then build each one in a separate build directory, never inside its source tree:

build-cross.sh (outline; replace X.Y with the versions you downloaded)

```
export PREFIX="$HOME/opt/cross"
export TARGET=i686-elf
export PATH="$PREFIX/bin:$PATH"

mkdir -p ~/src/build-binutils && cd ~/src/build-binutils
../binutils-X.Y/configure --target=$TARGET --prefix="$PREFIX" \
    --with-sysroot --disable-nls --disable-werror
make -j"$(nproc)" && make install

mkdir -p ~/src/build-gcc && cd ~/src/build-gcc
../gcc-X.Y.Z/configure --target=$TARGET --prefix="$PREFIX" \
    --disable-nls --enable-languages=c --without-headers
make -j"$(nproc)" all-gcc all-target-libgcc
make install-gcc install-target-libgcc
```

Add `export PATH="$HOME/opt/cross/bin:$PATH"` to `~/.bashrc`. The GCC build takes 20–40 minutes.

If the cross build fails

You can start Phase 1 with the host compiler while you fix it, as LOB does. Use these flags and link with `ld -m elf_i386`. Switch to the cross compiler before Phase 3, because the host compiler's hidden assumptions cause bugs that are very hard to recognise.

fallback CFLAGS for the host gcc

```
-m32 -ffreestanding -fno-pie -fno-stack-protector -fno-builtin -nostdlib -fcf-protection=none
```

Verify: `i686-elf-gcc --version` and `i686-elf-ld --version` work in a fresh terminal.

Task 0.4 Create the repository skeleton

Create the project layout, initialise Git, and ignore build outputs. Commit early and often from now on: small commits let you bisect your way back to the last working kernel when something breaks.

terminal

```
cd Turk-OS                    # your project folder
mkdir -p boot kernel lib include tests tools user
git init
printf 'build/\n*.o\n*.elf\n*.bin\n*.iso\n*.img\n' > .gitignore
git add . && git commit -m "Turk-OS: empty skeleton and study plan"
```

Verify: `git log` shows your first commit.

Task 0.5 Write the C warm-up library, and test it on Linux

Write these functions from scratch, without the C library, in `lib/`. They are small, and every one of them reappears in the kernel. Compile them with the *host* compiler for now and test them with `assert`, using the address and undefined-behaviour sanitizers to catch memory mistakes while you still have an OS to catch them.

File	Functions	Used again in
`string.c`	`memset`, `memcpy`, `memmove`, `memcmp`, `strlen`, `strcmp`, `strncpy`	Phase 2 (GCC may call these itself)
`convert.c`	`utoa(value, buf, base)` for bases 10 and 16, `itoa` with sign	Phase 2 (`kprintf`)
`bitmap.c`	`bitmap_set`, `bitmap_clear`, `bitmap_test`, `bitmap_find_first_clear`	Phase 5 (frame allocator)
`ringbuf.c`	fixed-size byte ring buffer: `rb_put`, `rb_get`, `rb_empty`, `rb_full`	Phase 4 (keyboard)
`list.h`	intrusive doubly linked list: `list_add`, `list_del`, `list_for_each`	Phases 7–8 (heap, tasks)

terminal

```
gcc -std=gnu11 -Wall -Wextra -g -fsanitize=address,undefined \
    -Iinclude tests/test_lib.c lib/*.c -o build/test_lib && ./build/test_lib
```

Verify: All asserts pass with sanitizers enabled, including edge cases: zero-length copies, overlapping `memmove`, `utoa(0)`, `itoa(INT_MIN)`, a full ring buffer, the last bit of a bitmap.

Task 0.6 Read your first assembly

You will learn to write assembly in Phase 1, but you can start reading it today. Write three tiny functions (add two ints, sum an array, call another function) and look at what the compiler makes of them. Compare `-O0` with `-O2`, and keep the output in your journal as a phrase book.

terminal

```
i686-elf-gcc -std=gnu11 -ffreestanding -O0 -S -masm=intel add.c -o add.s  # read add.s
i686-elf-gcc -std=gnu11 -ffreestanding -O0 -c add.c -o add.o
i686-elf-objdump -d -M intel add.o     # machine code next to the instructions
i686-elf-readelf -S add.o              # the sections: .text .data .bss ...
i686-elf-nm add.o                      # the symbols the linker will see
```

Verify: You can point to where each function starts, where its argument is read from the stack, and which register holds the return value.

Task 0.7 Smoke-test QEMU and GDB

Start the emulated PC with nothing to boot: `qemu-system-i386`. You should see the SeaBIOS firmware try every device and report that there is no bootable disk. That screen is the *exact* point where your code will take over in Phase 1. Also open the QEMU monitor (`Ctrl+Alt+2` in the window, or start with `-monitor stdio`) and run `info registers`.

Verify: You can find the value of `EIP` and `CR0` in the `info registers` output.

Task 0.8 Start the lab journal

Create `docs/journal.md`. For every session, write down the date, what you tried, what happened, and what you learned. When a bug takes more than an hour, write down the hypothesis before each attempt. This feels slow and saves weeks.

5 Testing and debugging

Most problems in this phase are environment problems. The table covers the common ones.

Symptom	Likely cause	What to do
`nasm: command not found` in VS Code, but it works in a normal terminal	Flatpak sandbox (Task 0.2)	Use the host terminal profile or a normal terminal.
GCC build stops with missing `gmp.h` or `mpfr.h`	Build dependencies missing	Install `gmp-devel mpfr-devel libmpc-devel`, rerun `configure`.
`configure: error: building in source directory`	Built inside the source tree	Use a separate `build-gcc` directory.
`i686-elf-gcc` found in one terminal, not another	`PATH` set only in that shell	Put the export in `~/.bashrc`; open a new terminal.
Sanitizer reports `heap-buffer-overflow` in your `strncpy`	Off-by-one on the terminator	Good: this is why you test on Linux first. Fix it now, not inside a kernel.

6 Milestone

Milestone: Workshop ready

- ☐ `i686-elf-gcc`, `i686-elf-ld`, `nasm`, `qemu-system-i386` and `gdb` run from your editor's terminal.
- ☐ The Turk-OS repository exists with the skeleton folders and at least one commit.
- ☐ The warm-up library passes its tests under AddressSanitizer and UBSan.
- ☐ You have compiled C to assembly and can explain the output of a small function.
- ☐ You have seen the SeaBIOS "no bootable device" screen and used `info registers`.
- ☐ `docs/journal.md` has its first entries.

Self-check questions

1. What are the three big ideas of OSTEP, and which Turk-OS phases correspond to each?
2. What is the difference between kernel mode and user mode, and who enforces it?
3. Which C headers are still available in freestanding mode, and why exactly those?
4. What can the compiler do to a non-`volatile` pointer access that would break a device driver?
5. Why does the `.bss` section take no space in the ELF file? Who zeroes it?
6. Round `0x00123456` up to the next 4 KiB boundary by hand.
7. Name two things the host GCC assumes that are false inside a kernel.

7 Going further

If you finish early, set up `clangd` or the VS Code C/C++ extension with a `compile_commands.json` so the editor understands your cross-compiled code (`bear -- make` generates the file). Work through the Makefile chapter of the GNU Make manual, especially automatic variables (`$@`, `$<`, `$^`) and pattern rules. Then read MOS §1.4 *The Operating System Zoo* for the wider landscape.

Next: **Phase 1 x86 Assembly and the Boot Process** →

Books used in this plan

LOB — Erik Helin, Adam Renberg. *The little book about OS development*, 2015. The hands-on x86 guide. Free at `https://littleosbook.github.io`.

OSTEP — Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*, v0.90, Arpaci-Dusseau Books, 2015. Main theory text. Free at `https://pages.cs.wisc.edu/~remzi/OSTEP/`.

OSC — A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. Reference theory text.

MOS — Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, 4th ed., Pearson, 2015. Hardware, I/O, design, security.

OSDI — Andrew S. Tanenbaum, Albert S. Woodhull. *Operating Systems: Design and Implementation*, 3rd ed., Pearson Prentice Hall, 2006. MINIX 3 with full source listing.

LKD — Robert Love. *Linux Kernel Development*, 3rd ed., Addison-Wesley, 2010. How Linux implements each subsystem.

LKP — Kaiwan N. Billimoria. *Linux Kernel Programming*, Packt, 2021. Modern kernel practice: logging, allocators, the scheduler, synchronisation.

The books do not come with this plan. LOB and OSTEP are free online; buy or borrow the other five (the roadmap lists their ISBNs). Where books disagree, the plan follows the newer one when its method is also better for Turk-OS. Page numbers are the *printed* page numbers of these editions, not PDF page indices. With another edition, or with the free OSTEP chapters (each one starts at page 1), find the reading by its chapter and section numbers. MINIX source references like `kernel/proc.c (07400)` give the file and its starting line in OSDI Appendix B.